



In Class Assignment (Graded 2)

Python Functions, Packages, Scope & Exceptions - In-Class Assignment

Submission Instructions:

- Create **one Python file** (e.g., `assignment.py`) containing all your code for this assignment.
- Submit this single file before the class on the **coming Tuesday August 4th**.

General Instructions: Complete all the tasks below using a Python script. Use `print()` statements to display the output after each task to verify your code works. Keep the code simple, readable, and include comments where helpful.

Part 1: Functions Without Parameters

1. Write a function called `greet_user` that prints: `Hello, welcome to Python class!`
2. Write a function called `show_python_tip` that prints: `Tip: Use clear variable names in your code.`
3. **Test it:** Call both functions to display their messages on the screen.

Part 2: Functions With Parameters

1. Write a function called `greet_by_name(name)` that takes a name as an argument and prints: `Hello <name>, nice to meet you!`

(Example: `greet_by_name("Alex")` should print: Hello Alex, nice to meet you!)

2. Write a function called `add_numbers(a, b)` that **returns** the sum of two numbers. *(Example: `add_numbers(3, 5)` should return 8)*
3. **Test it:** Call `greet_by_name` with your own name. Call `add_numbers`, save the result in a variable, and print that variable.

Part 3: Using Built-in Packages

1. Use the math module:

- Import the math module at the top of your file.
- Write a function called `find_square_root(number)` that returns the square root of the number using `math.sqrt()`.
(Example: `find_square_root(16)` → 4.0)

2. Use the random module:

- Import the random module at the top of your file.
- Write a function called `roll_dice()` that returns a random number between 1 and 6 using `random.randint()`.

3. **Test it:** Call both functions and print their results. *(Example output for dice: You rolled a 4)*

Part 4: Variable Scope (Local vs. Global)

1. Create a **global variable** called `coding_language` and set it to "Python".
2. Write a function called `demonstrate_scope()`:
 - Inside the function, create a **local variable** with the exact same name (`coding_language`) and set it to "JavaScript".

- Print the local variable from inside the function. (*Example: Inside function: JavaScript*)

3. Test it:

- Call `demonstrate_scope()`.
- Print the `coding_language` variable *outside* the function to prove that the global variable hasn't changed. (*Example: Outside function: Python*)

Part 5: Exception Handling (try / except / finally)

1. Write a function called `safe_divide(num1, num2)`.
2. Inside the function, use a **try** block to attempt to divide `num1` by `num2` and print the result.
3. Use an **except ZeroDivisionError:** block to catch the error if someone tries to divide by zero. If this happens, print: `Error: You cannot divide by zero!`
4. Use a **finally** block that prints: `Division attempt completed.` (This should print whether the division succeeded or failed).
5. Test it:
 - Call `safe_divide(10, 2)` (This should succeed).
 - Call `safe_divide(10, 0)` (This should trigger the except block).

0 % 0 of 1 topics complete

in Class Assignment Graded 2

 Assignment